

SCS 1309

Database Management Systems

Lecture 8 – Normalization

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

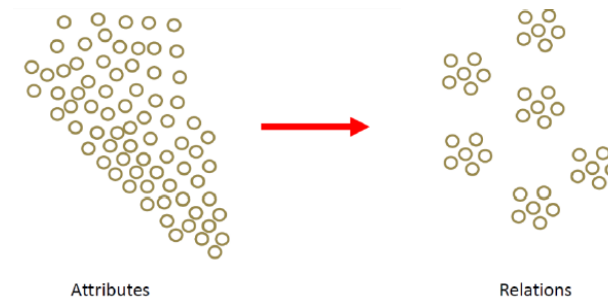
Outline of this lecture

1. What is normalization
2. Data Redundancy
3. Update Anomalies
4. Key types
5. Functional Dependencies
6. Process of Normalization
 - Unnormalized Form (UNF)
 - First Normal Form (1NF)
 - Second Normal Form (2NF)
 - Third Normal Form (3NF)

1. What is Normalization?

Normalization

- Normalization is a technique for producing a set of relations with desirable properties, given the data requirements of an enterprise.
- Normalization **begins** by examining the relationships (or functional dependencies) between attributes.
 - Attributes describe some property of the data or of the relationships between the data.



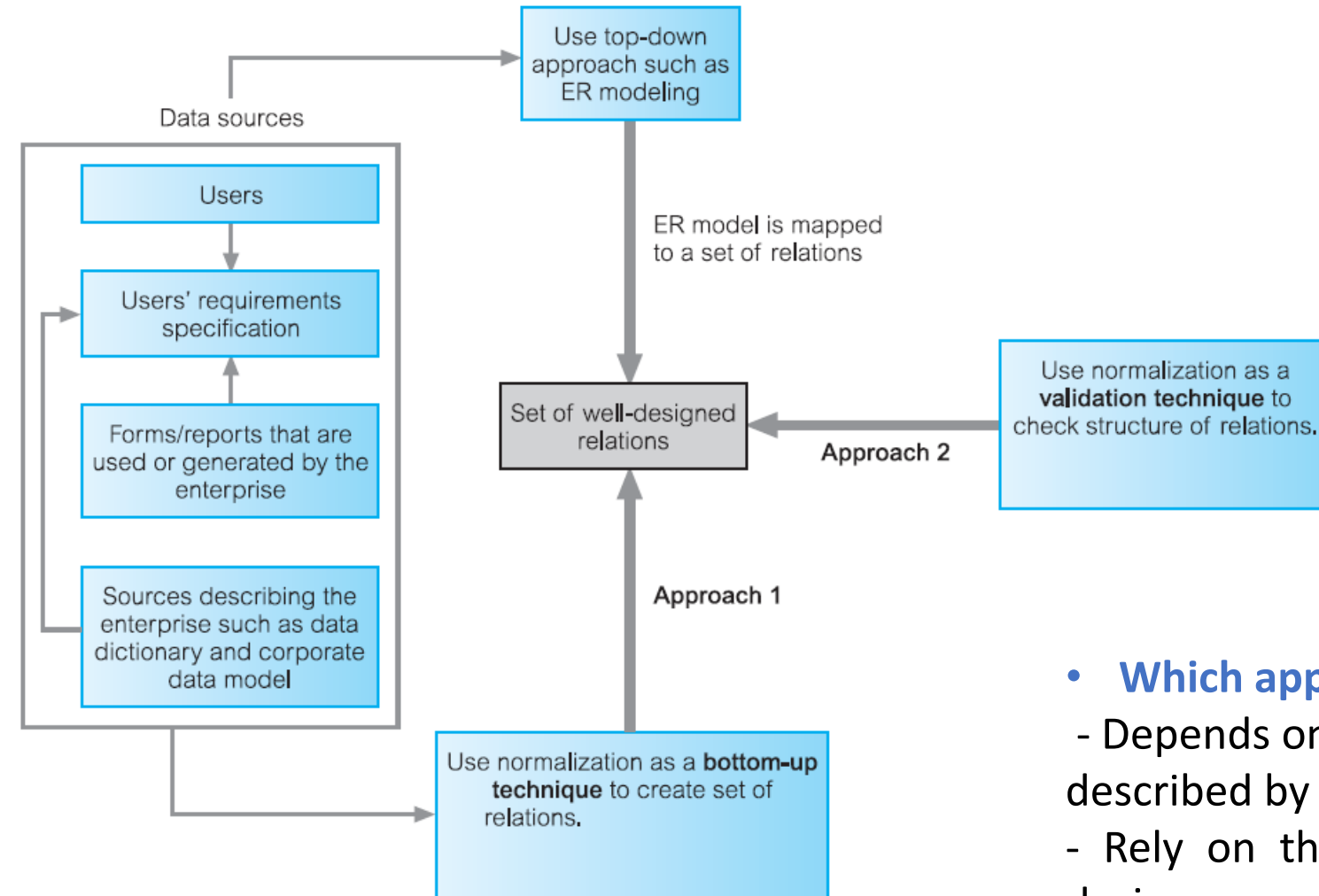
- **Next**, organizing database data by minimizing redundancy and eliminating anomalies such as update, insertion, and deletion issues.

Purpose of Normalization

- Identify a suitable set of relations that support the data requirements of an enterprise.
- The characteristics of a suitable set of relations:
 - the minimal number of attributes necessary to support the data requirements.
 - attributes with a close logical relationship (functional dependency) are found in the same relation.
 - minimal redundancy with each attribute represented only once, with the important exception of attributes that form all or part of foreign keys, which are essential for the joining of related relations.

Benefit: The database will be easier for the user to access and maintain the data, and take up minimal storage.

How Normalization Supports Database Design?



- **Approach 1** is often **limited** by the level of detail that the database designer is reasonably expected to manage.
- **Approach 2**, the database designer focuses on only part of the database, such as a single relation, at any one time.

- **Which approach to follow?**

- Depends on the size and complexity of the database being described by the data sources.
- Rely on the preference and expertise of the database designer.

2. Data Redundancy

What is data redundancy ?

- A major aim of relational database design is to **group attributes into relations** to **minimize data redundancy**.
 - Redundancy occurs when the same piece of data is stored in multiple places within a database.
- The potential benefits of resolving redundancy:
 - **updates to the data stored in the database are achieved with a minimal number of operations**, thus reducing the opportunities for data inconsistencies
 - **reduction in the file storage space** required by the base relations, thus minimizing costs.
- Of course, relational databases also rely on the existence of **a certain amount of data redundancy**.
 - This redundancy is in the form of copies of primary keys (or candidate keys) acting as foreign keys in related relations to enable the modeling of relationships between data.

Problems associated with data redundancy

Activity 1: Compare the **Staff** and **Branch** relations with the **StaffBranch** relation.

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

StaffBranch relation: there is redundant data; the details of a branch are repeated for every member of staff located at that branch.

Branch relation: branch details appear only once for each branch in the Branch relation.

Staff relation: only the branch number (branchNo) is repeated in the Staff relation to represent where each member of staff is located.

Relations that have redundant data may have problems called **update anomalies**.

3. Update Anomalies

Insertion, Deletion, and Modification anomalies

Insertion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Task: Insert the details of new members of staff located at branch number B007 into the **StaffBranch** relation.

- Here, you must enter the correct details of branch number B007 so that the branch details are consistent with values for branch B007 in other tuples of the **StaffBranch** relation.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

But,

The **Staff** and **Branch** relations do not suffer from this potential inconsistency because you enter only the appropriate branch number for each staff member in the **Staff** relation.

Instead, the details of branch number B007 are recorded in the database as a single tuple in the **Branch** relation.

Insertion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Insert details of a new branch that currently has no members of staff into the **StaffBranch** relation.

- It is necessary to enter *nulls* into the attributes for staff, such as *staffNo*. However, as *staffNo* is the primary key for the **StaffBranch** relation, attempting to enter nulls for *staffNo* violates **entity integrity** and is not allowed.

But,

The **Staff** and **Branch** relations avoid this problem because branch details are entered in the **Branch** relation separately from the staff details.

The details of staff located at that branch are entered at a later date into the **Staff** relation.

Insertion anomaly - Summary

- An insertion anomaly occurs when data cannot be inserted into a database due to other missing data.
- **Common Cause:**
 - When a foreign key must not be *NULL*, but lacks the appropriate data.
 - The required referenced data does not yet exist.
 - Database constraints prevent the insertion.
- **Why it happen?**
 - Foreign key dependency is not satisfied
 - Incorrect order of operations when inserting related data
 - Required parent record is missing
- Insertion anomaly itself is a form of data integrity problem.
- It doesn't directly cause redundancy (duplicated data), but rather a failure to insert data due to missing dependencies.

Deletion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Delete a tuple from the **StaffBranch** relation that represents the member of staff *Mary Howe* (staff number SA9).

- When you delete the tuple for staff number SA9 from the **StaffBranch** relation, the details relating to branch number B007 are lost from the database.

But,

The **Staff** and **Branch** relations avoid this problem, because branch tuples are stored separately from staff tuples and only the attribute *branchNo* relates the two relations.

- If you delete the tuple for staff number SA9 from the **Staff** relation, the details on branch number B007 remain unaffected in the **Branch** relation.

Deletion anomaly - Summary

- A deletion anomaly occurs when data is unintentionally lost due to the deletion of other data.
- **Impact on Data Integrity**
 - Causes unintended data loss
 - Leads to loss of important information
 - Creates data inconsistency
- It doesn't directly cause redundancy (duplicated data), but rather results in loss of necessary data.

Modification anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Task: Change the value of the address attribute of branch number B003 in the **StaffBranch** relation.

- If this modification is not carried out on all the appropriate tuples of the **StaffBranch** relation, the database will become inconsistent.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

The **Staff** and **Branch** relations have more desirable properties than the **StaffBranch** relation.

Modification anomaly - Summary

- A modification anomaly occurs when a change to data **requires multiple updates in different rows** of a table (when **data is only partially updated in a database**).
- This can cause problems as we then need to spend time searching for and updating each reference to the data element.
- **Impact on Data Integrity**
 - Causes data inconsistency
 - Leads to conflicting information
 - Increases risk of human error

We can avoid these insertion, deletion, and modification anomalies by decomposing the original relation into the appropriate sub-relations.

Decomposition of a larger relation into smaller relations

Decomposition should have the following two **properties**:

- The **lossless-join** property ensures that any instance of the original relation can be identified from corresponding instances in the smaller relations.
- The **dependency preservation** property ensures that a constraint on the original relation can be maintained by simply enforcing some constraint on each of the smaller relations.
- In other words, we do not need to perform joins on the smaller relations to check whether a constraint on the original relation is violated (Or Simply preserve the associated set of functional dependencies).

Example: Although the **StaffBranch** relation is subject to update anomalies, we can avoid these anomalies by decomposing the original relation into the **Staff** and **Branch** relations.

Example 1: lossless-join

Let us populate R with sample data and try the experiment;

A	B	C	D
a ₁	a ₂	a ₃	a ₄
a ₁	a ₄	a ₃	a ₂

According to the decomposition, we shall get R₁ and R₂ as follows;

R ₁		
A	B	C
a ₁	a ₂	a ₃
a ₁	a ₄	a ₃

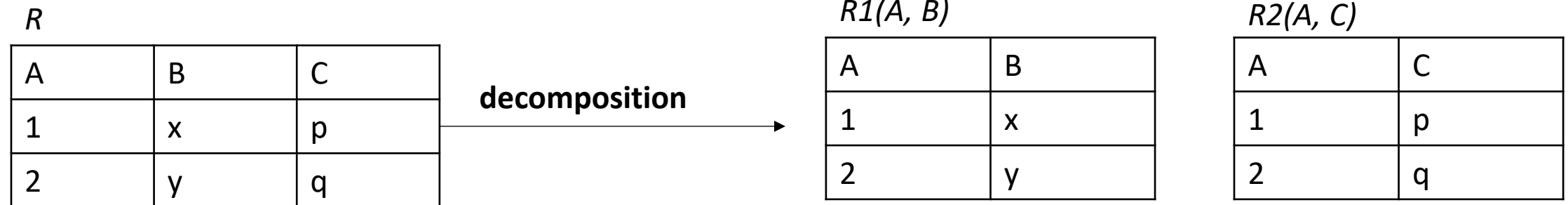
R ₂		
A	C	D
a ₁	a ₃	a ₄
a ₁	a ₃	a ₂

Join back R₁ and R₂ must result in R if the decomposition is lossless.

R ₁			⋈	R ₂			=	R'			
A	B	C	⋈	A	C	D	=	A	B	C	D
a ₁	a ₂	a ₃		a ₁	a ₃	a ₄		a ₁	a ₂	a ₃	a ₄
a ₁	a ₄	a ₃		a ₁	a ₃	a ₂		a ₁	a ₂	a ₃	a ₂
								a ₁	a ₄	a ₃	a ₂
								a ₁	a ₄	a ₃	a ₄

R' is the result of natural join of R₁ and R₂, and R' is not equal to R the base relation. Hence, **the decomposition is not lossless join decomposition.**

Example 2: lossless-join



This is exactly the original table.
Lossless Join is satisfied.

Next, Join $R_1 \bowtie R_2$

R

A	B	C
1	x	p
2	y	q

4. Key types

Key Types

Super key

- A super key is an attribute, or set of attributes, that identifies tuples of a relation uniquely, while a **candidate key** is a **minimal super key**.

Candidate Key

- A candidate key is a **minimal set of attributes** of an entity that **uniquely identifies each occurrence** of that entity.
- You may identify **more than one candidate key**, in which case you must choose one to be the **primary key**; the remaining candidate keys are called **alternate keys**.

Activity 2:

Consider the relation *Students* (*studentID*, *studentRollNo*, *Address*, *Marks*)

Identify the candidate keys.

studentID and *studentRollNo*

Identify the primary key and alternate key.

primary key: *studentID*, alternate key: *studentRollNo*

Key Types

Primary Key

- This is a **candidate key** that is selected to identify tuples uniquely within the relation.
- The value of the primary key **cannot be repeated** inside a table and **cannot hold null** values.
- This is the **minimal identifier** that is used to identify tuples uniquely.
- Generally **first column** is defined as the primary key.
- The candidate keys that are not selected to be the primary key are called **alternate keys**.

Activity 3: The **staffNo** and National Identity Card (**NIC**) number can be considered as the two candidate keys for the *Staff* entity. Which one must be selected as the primary key?

- Consider the attribute length, the minimal number of attributes required, and the future certainty of uniqueness.

Key Types

Composite primary key

- A candidate key that consists of **two or more attributes**.

Example: Consider the composite primary key of the *Viewing* relation, comprising the client number (**clientNo**) and the property number (**propertyNo**).

Viewing (clientNo, propertyNo, viewDate, comment)

- Users should not be able to insert a tuple into the *Viewing* relation with either a *null* for the **clientNo** attribute, or a *null* for the **propertyNo** attribute, or *nulls* for both attributes.

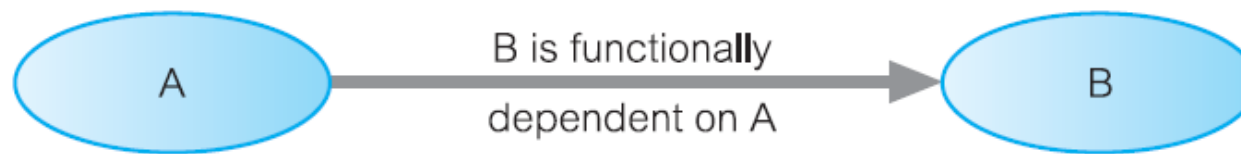
Except for these keys, there are foreign keys, compound keys, and surrogate keys...

5.Functional Dependency

What is Functional Dependency?

- **Functional Dependency (FD)** is a constraint that determines the **relation of one attribute to another attribute** in a DBMS.
- An FD is denoted by an arrow “ $\rightarrow$ ”.

Example: if A and B are attributes of relation R, **B is functionally dependent on A** (denoted $A \rightarrow B$), if each value of A is associated with exactly one value of B.



(A and B may each consist of one or more attributes).

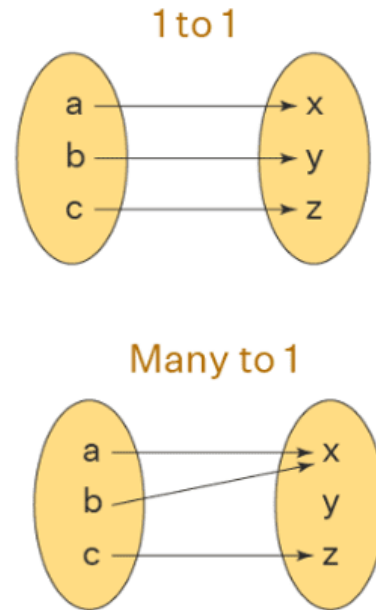
- An alternative ways: '**A functionally determines B**' or '**A is the determinant of B**'.

What is Functional Dependency?

- An FD holds a **one-to-one** relationship between the attribute(s) on the left-hand side and those on the right-hand side.
 - The relationship in the opposite direction can be a **one-to-one** or **one-to-many** relationship.
- The **attribute or group of attributes** on the left-hand side of the arrow is called the **determinant**.

Example: Consider the functional dependency $A \rightarrow B$

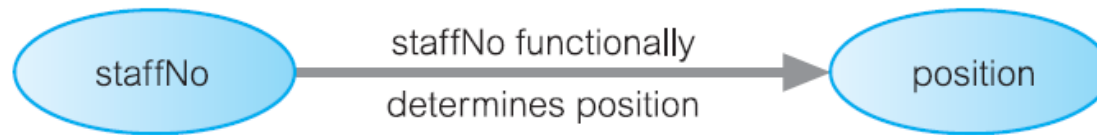
- For any given value of A, there can be only one value of B across all tuples at any moment in time.
- When two tuples have the same value of A, they also have the same value of B.
- However, for a given value of B, there may be several different values of A.



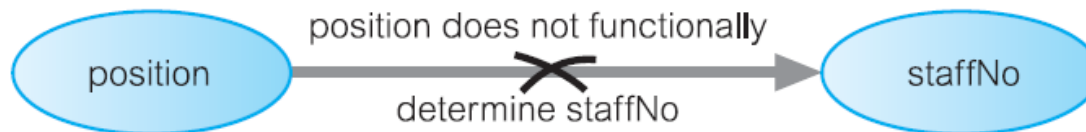
Example

Consider the attributes *staffNo* and *position* of the *Staff* relation.

- For a specific *staffNo* (**SL21**), you can determine the *position* of that member of staff as **Manager** (*staffNo* functionally determines *position*).



- However, the opposite is not true, as *position* does not functionally determine *staffNo*.



Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Example (Cont...)

- A member of staff holds one **position**; however, there may be several members of staff with the same **position**.

Example: Staff numbers SL21 and SG5 are managers.

- The relationship between **staffNo** and **position** is one-to-one (1:1): for each staff number, there is only one position.
- The relationship between **position** and **staffNo** is one-to-many (1:*) : there are several staff numbers associated with a given position.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Full functional dependency

- A **full functional dependency** occurs when you already meet the requirements for a functional dependency.
- Consider the functional dependency $A \rightarrow B$, then B is fully functionally dependent on A, **but not on any proper subset of A**.
 - The **determinants** should have the **minimal number of attributes** necessary to maintain the FD with the attribute(s) on the right-hand side.
 - All non-key attributes depend on the key attribute.
 - The removal of any attribute from A results in the dependency no longer existing.
 - If there is a single determinant, it automatically becomes a full functional dependency.

Full functional dependency

Example:

Consider the functional dependency $\{\text{staffNo}, \text{sName}\} \rightarrow \text{position}$ exists in the *Staff* relation, but it is not a full functional dependency because you can remove *sName* from the left side of the statement without impacting the dependency relationship.

Activity 4

Consider the functional dependency $\text{staffNo}, \text{sName} \rightarrow \text{branchNo}$ that exists in the *Staff* relation. Is this a full functional dependency?

- Each value of (staffNo, sName) is associated with a single value of *branchNo*.
- However, it is not a full functional dependency because *branchNo* is also functionally dependent on a subset of (staffNo, sName), namely staffNo.
- The full functional dependency is $\text{staffNo} \rightarrow \text{branchNo}$

Partial functional dependency

- A functional dependency $A \rightarrow B$ is a **partial dependency** if there is some attribute that can be removed from A and yet the dependency still holds.

Example:

$\{\text{staffNo}, \text{sName}\} \rightarrow \text{position}$

$\{\text{staffNo}, \text{sName}\} \rightarrow \text{branchNo}$

- A partial dependency is when you have a **composite primary key**, and non-key attributes functionally dependent on one, but not all of the attributes that make up the composite primary key.
- It is possible to derive full functional dependency from partial dependency.

$\text{staffNo} \rightarrow \text{position}$

$\text{staffNo} \rightarrow \text{branchNo}$

Activity 5

Consider the attributes in the **StaffBranch** relation.

Assumption: The **position** held and the **branch** determine a member of staff's **salary**.

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Identify the functional dependencies.

staffNo $\rightarrow$ sName, position, salary, branchNo, bAddress

branchNo $\rightarrow$ bAddress

bAddress $\rightarrow$ branchNo

branchNo, position $\rightarrow$ salary

bAddress, position $\rightarrow$ salary



Based on the given assumption

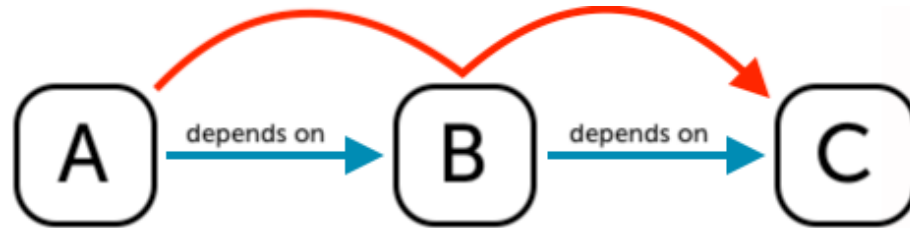
Types of functional dependencies

Apart from the **Fully Functional** and **Partial Functional** dependencies, the following types of functional dependencies exist in DBMS:

- Transitive Dependency
- Trivial and Non-Trivial Dependency
- Multivalued Dependency

Transitive Dependency

- The condition where A, B, and C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).



- Transitive Dependency occurs because one non-key attribute is dependent on other non-key attributes.
- Transitive dependencies occur when there is an indirect relationship that causes a functional dependency.

Transitive Dependency

Example: Consider the following functional dependencies within the *StaffBranch* relation.

$\text{staffNo} \rightarrow \text{sName, position, salary, branchNo, bAddress}$

$\text{branchNo} \rightarrow \text{bAddress}$

- The transitive dependency $\text{branchNo} \rightarrow \text{bAddress}$ exists on staffNo via branchNo .
- In other words, the staffNo attribute functionally determines the bAddress via the branchNo attribute, and neither branchNo nor bAddress functionally determines staffNo .

Multi-Valued Dependency (MVD)

- Multi-Valued Dependency represents a dependency between attributes (for example, A, B, and C) in a relation, such that **for each value of A, there is a set of values for B and a set of values for C**.
- However, the sets of values for **B and C are independent** of each other.

Example: Consider the **BranchStaffOwner** relation, which displays the names of members of staff (**sName**) and property owners (**oName**) at each branch office (**branchNo**).

- Assume that **sName** uniquely identifies each member of staff and that the **oName** uniquely identifies each owner.
- **Ann Beech** and **David Ford** work at branch **B003**, and property owners called **Carol Farrel** and **Tina Murphy** are also registered at the same branch.
- However, there is no direct relationship between **sName** and **oName** at a given branch office.

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Multi-Valued Dependency (MVD)

- You must create a tuple for every combination of `sName` and `oName` to ensure that the relation is consistent.

`branchNo →> sName`

`branchNo →> oName`

- This constraint represents a multi-valued dependency in the `BranchStaffOwner` relation.

- Here, an MVD exists because two independent 1:* relationships are represented in the `BranchStaffOwner` relation.
- An MDV can be further defined as being **trivial** or **nontrivial**.

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Trivial and Nontrivial Dependencies

- An MVD $A \twoheadrightarrow B$ in relation R is defined as being **trivial** if,
 - (a) **B is a subset of A ($B \subseteq A$)** or
 - (b) **$A \cup B = R$**
- A MVD is defined as being **nontrivial** if neither (a) nor (b) is satisfied.
Example: $\{A, B\} \twoheadrightarrow A$ (Trivial, because A is a subset of $\{A, B\}$)
- A trivial MVD does not specify a constraint on a relation, while a nontrivial MVD does specify a constraint.
- The MVD in the **BranchStaffOwner** relation is nontrivial as neither condition (a) nor (b) is true for this relation.

Trivial and Nontrivial Dependencies

Example: Add a new property owner for branch B003.

- You have to create two new tuples, one for each member of staff, to ensure that the relation remains consistent.
- This is an example of an **update anomaly** caused by the presence of the nontrivial MVD.
- The relation remains poorly structured, due to the data redundancy caused by the presence of the nontrivial MVD.

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Activity 6 - Using sample data

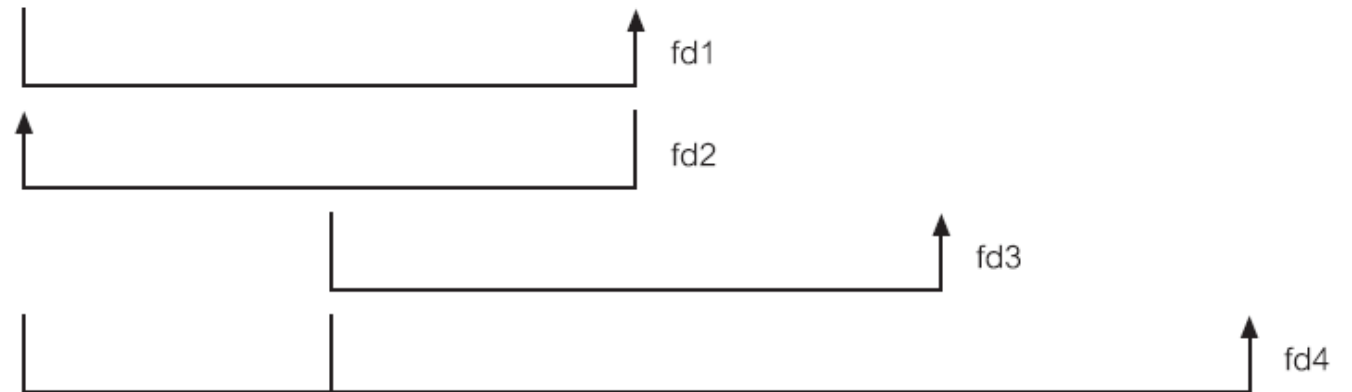
Consider the Sample relation displaying data for attributes **A, B, C, D, and E**.

Identify the functional dependencies that exist between these attributes.

List down functional dependencies between attributes A to E in the Sample relation.

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t



$A \rightarrow C$ (fd1)

$C \rightarrow A$ (fd2)

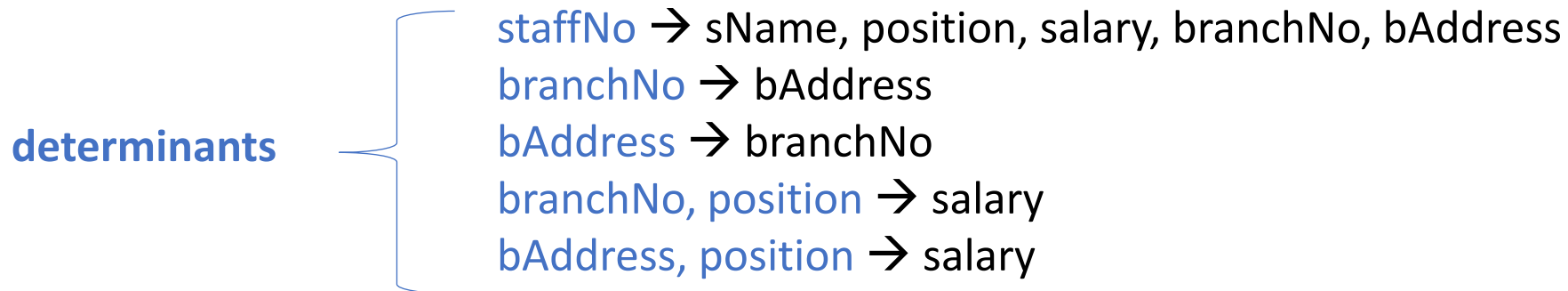
$B \rightarrow D$ (fd3)

$A, B \rightarrow E$ (fd4)

Identifying the Primary Key for a Relation using Functional Dependencies

- Functional dependencies for a relation specify the set of **integrity constraints** that must hold on a relation.
 - An important integrity constraint to consider is the identification of candidate keys, one of which is selected to be the primary key for the relation.

Activity 7: Identify the candidate key(s) for the **StaffBranch** relation using the functional dependencies identified in Activity 5.



- The only candidate key and therefore the primary key is **staffNo**, as *all* other attributes of the relation are functionally dependent on staffNo.

Identifying the Primary Key for a Relation using Functional Dependencies

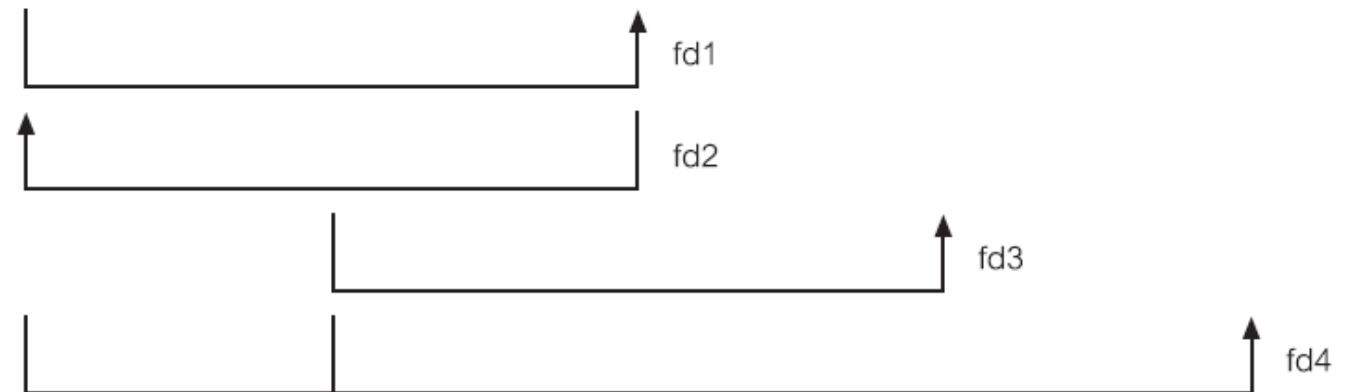
Activity 8:

Examine the determinant for each functional dependency to identify the candidate key(s) for the relation.

determinants {
 $A \rightarrow C$ (fd1)
 $C \rightarrow A$ (fd2)
 $B \rightarrow D$ (fd3)
 $A, B \rightarrow E$ (fd4)

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t



Identifying the Primary Key for a Relation using Functional Dependencies

Activity 8 (Cont.):

- A determinant that functionally determines all the other attributes of the relation is (A,B).
 - A functionally determines C
 - B functionally determines D
 - (A, B) functionally determines E.
- So, the attributes that make up the determinant (A, B) can determine all the other attributes in the relation, either separately as A or B or together as (A, B).
- This is not a characteristic of the other determinants in the Sample relation as in each case they can determine only one other attribute in the relation.
- Are there any other candidate keys ?

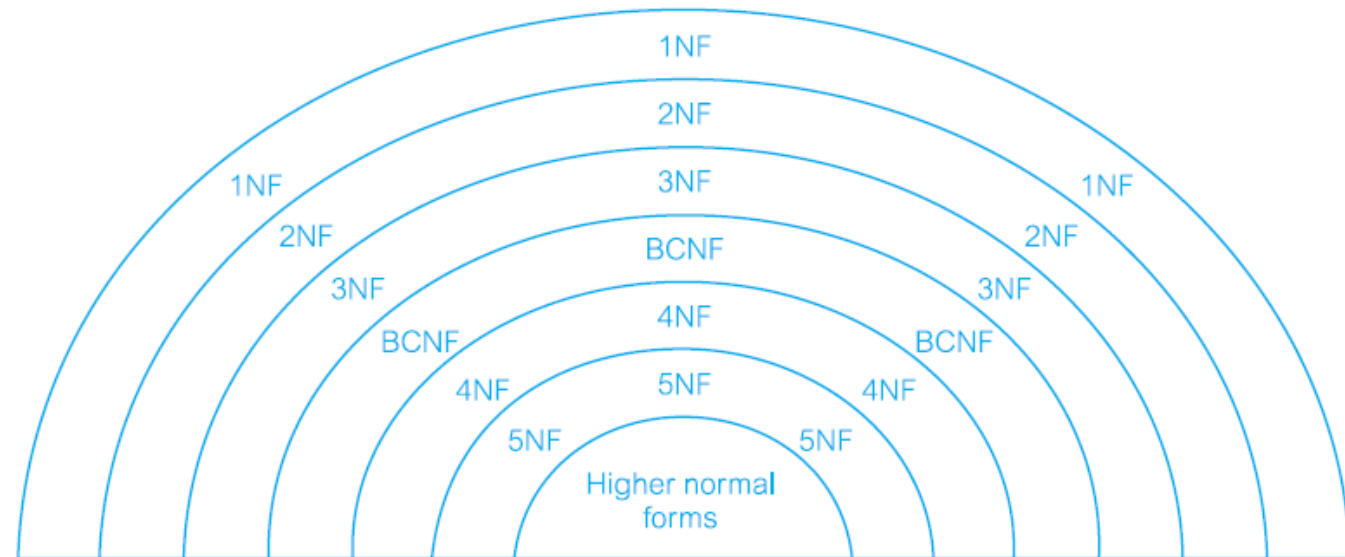
6. Process of Normalization

Introduction

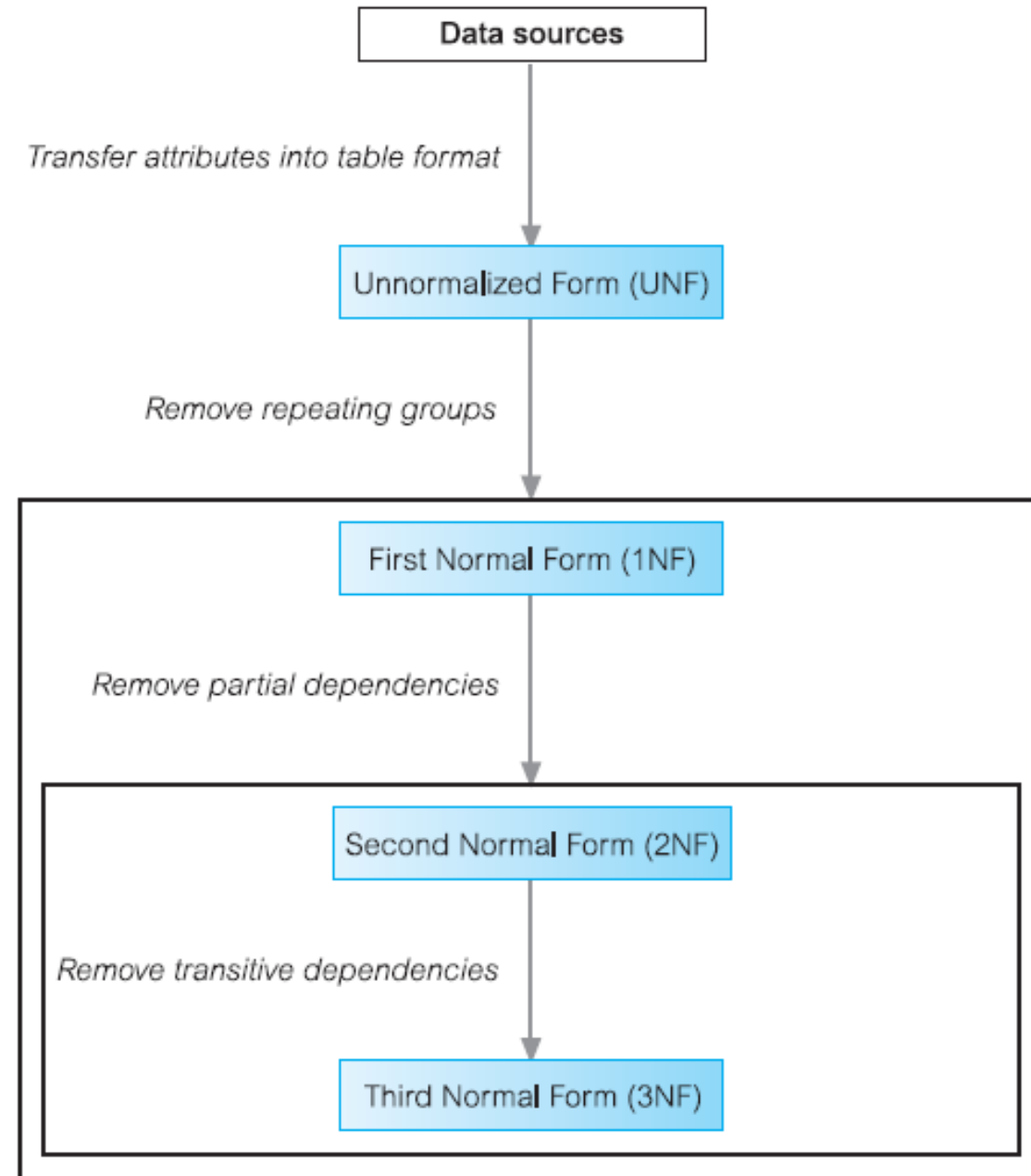
- Normalization is a formal technique for analyzing relations based on their **primary key (or candidate keys)** and **functional dependencies**.
- The technique involves a **series of rules (or normal forms)** that can be used to test individual relations so that a database can be normalized to any degree.
 - Identifies the **optimal grouping for the attributes**.
 - When a requirement in the rules is not met, the relation violating the requirement must be decomposed into relations that individually meet the requirements of normalization.
- As normalization proceeds, the relations become progressively **more restricted (stronger) in format** and also **less vulnerable to update anomalies**.
- But, **performance degrades**.

Relationship between the normal forms

- **Three main normal forms:** First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).
- Subsequently, R. Boyce and E.F. Codd introduced a stronger definition of 3NF called Boyce–Codd Normal Form (BCNF).
- Higher normal forms that go beyond BCNF were introduced later, such as 4NF and 5NF.
- However, these later normal forms deal with very rare situations.
- Figure shows that some 1NF relations are also in 2NF and that some 2NF relations are also in 3NF, and so on.



Process of Normalization



6.1. Unnormalized Form (UNF)

Unnormalized Form (UNF)

- The process of normalization starts with first transferring the data from the sources into a table format with rows and columns.
- In this format, the table is in **Unnormalized Form** and is referred to as an **unnormalized table**.
 - Contains one or more **repeating groups** in the tables.
 - A **repeating group** is an attribute, or group of attributes, within a table that occurs with **multiple values for a single occurrence of the key attribute(s)** for that table.

Removing repeating groups

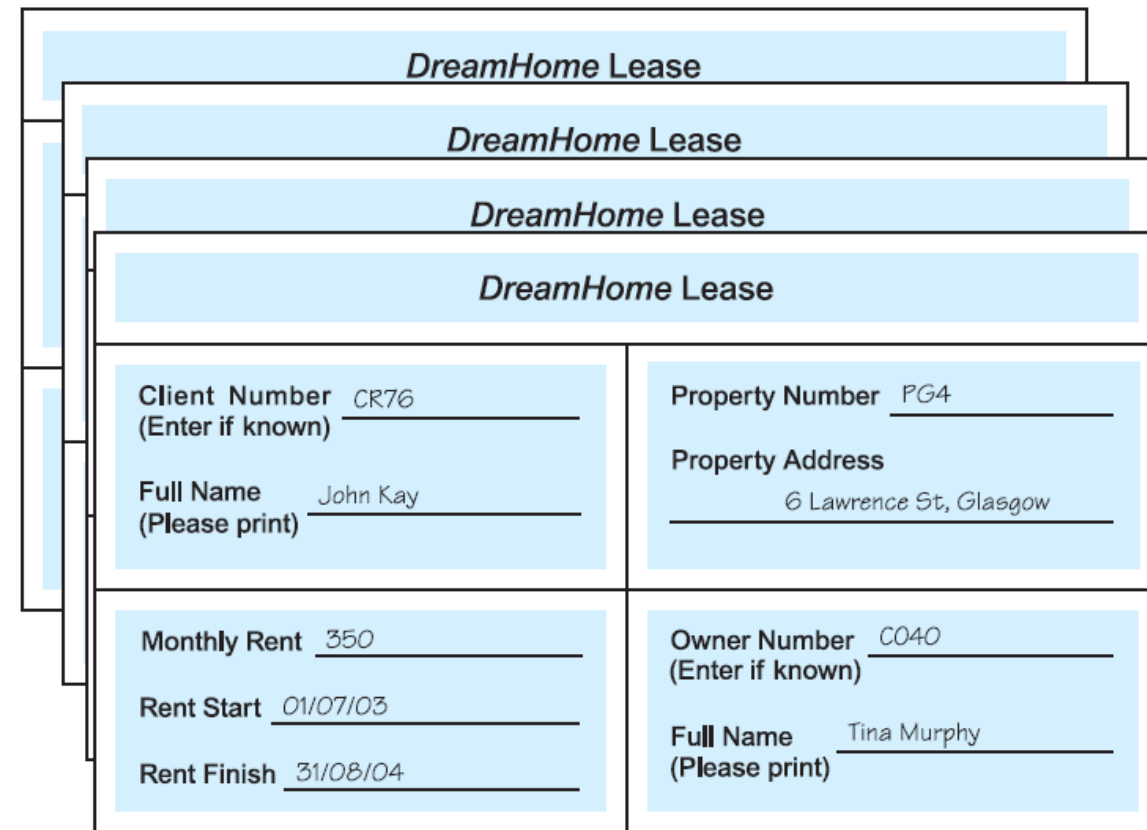
There are **two common approaches** to removing repeating groups from unnormalized tables.

1. Flattening the table - By entering appropriate data in the empty columns of rows containing the repeating data (fill in the blanks by duplicating the nonrepeating data, where required)
2. By placing the repeating data, along with a copy of the original key attribute(s), in a separate relation.
 - Sometimes the unnormalized table may contain **more than one repeating group**, or **repeating groups within repeating groups**.
 - In such cases, this approach is applied repeatedly until no repeating groups remain.

Example - UNF

A collection of (simplified) *DreamHome* leases →

Assumption: A client rents a given property only once and cannot rent more than one property at any one time.



DreamHome Lease

DreamHome Lease

DreamHome Lease

DreamHome Lease

Client Number CR76
(Enter if known)

Property Number PG4

Full Name John Kay
(Please print)

Property Address
6 Lawrence St, Glasgow

Monthly Rent 350

Rent Start 01/07/03

Rent Finish 31/08/04

Owner Number CO40
(Enter if known)

Full Name Tina Murphy
(Please print)

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← *ClientRental* unnormalized table

The key attribute for the *ClientRental* unnormalized table is **clientNo**.

6.2. First Normal Form (1NF)

First Normal Form (1NF)

1NF: A relation in which the intersection of each row and column contains **one and only one value**.

- The structure of the repeating groups from the previous example:

Repeating Group = (propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

- UNF to 1NF: ensure that there is a single value at the intersection of each row and column.

Approach 1: Flattening the table

- **Remove the repeating group** by entering the appropriate client data into each row.

Approach 2:

- **Remove the repeating group** by placing the repeating data, along with a copy of the original key attribute, in a separate relation.

Example – UNF to 1NF

Approach 1: Flattening the table

ClientRental unnormalized table →

Repeating Group

ClientRental								
clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← The resulting 1NF *ClientRental* relation

Example – UNF to 1NF

Approach 2: Introduce alternative 1NF relations.

ClientRental unnormalized table →

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

Repeating Group

PropertyRentalOwner

clientNo	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

← The resulting *Client* and *PropertyRentalOwner* relations are both in 1NF as there is a single value at the intersection of each row and column.

1NF

- The resulting tables from both previous approaches are now referred to as 1NF relations containing **atomic (or single) values** at the intersection of each row and column.
- **Approach 1** introduces **more redundancy** into the original UNF table, whereas **approach 2** creates **two or more relations** with **less redundancy** than in the original UNF table.

Thus, approach 2 moves the original UNF table further along the normalization process than approach 1.

- **No matter which initial approach is taken, the original UNF table will be normalized into the same set of relations at the end.**

6.3. Second Normal Form (2NF)

Second Normal Form (2NF)

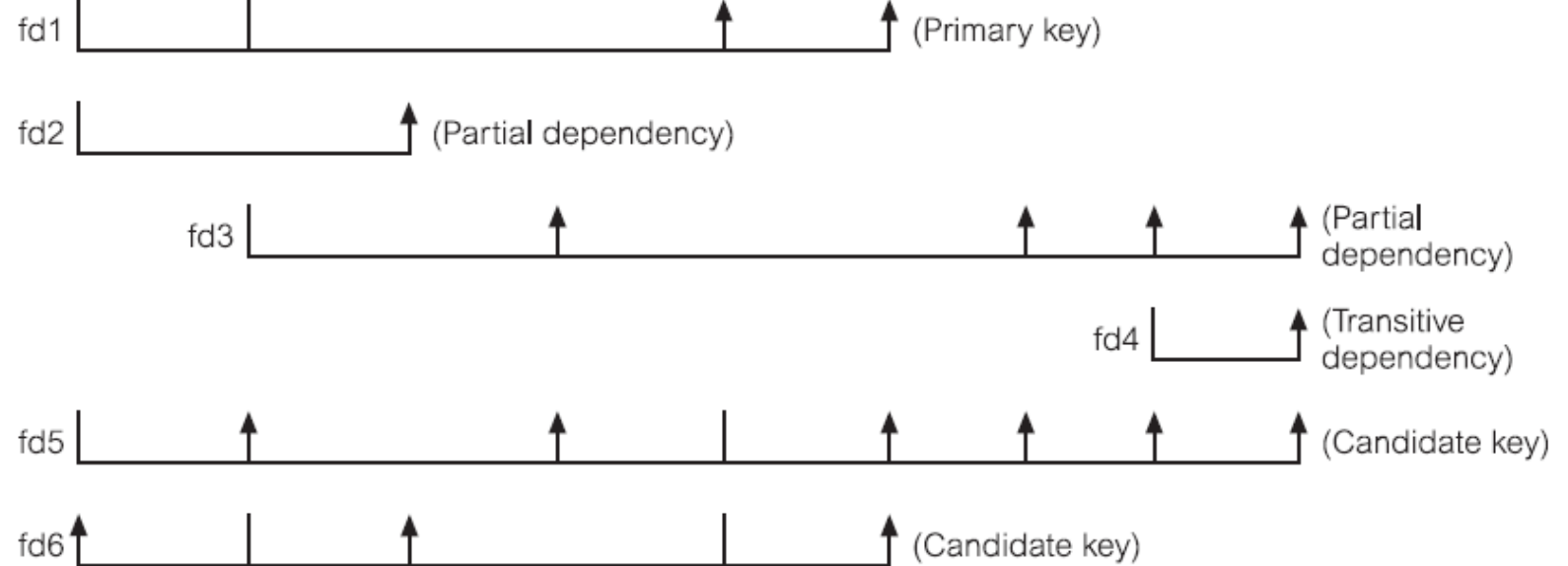
2NF: A relation that is in 1NF and every non-primary-key attribute is **fully functionally dependent** on the primary key.

- 2NF is based on the concept of **full functional dependency**.
- 2NF applies to relations with **composite keys** (relations with a primary key composed of two or more attributes).
- A relation with **a single-attribute primary key** is automatically in at least 2NF.
 - A relation that is not in 2NF may **suffer from the update anomalies**.
- The normalization of 1NF relations to 2NF involves the **removal of partial dependencies**.
 - If a partial dependency exists, **remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant**.

Example

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
----------	------------	-------	----------	-----------	------------	------	---------	-------



Functional dependencies (fd1 to fd6) for the *ClientRental* relation →

In this example, we assume that the *rentFinish* attribute is not appropriate as a component of a candidate key, as it may contain nulls.

- fd1 $\text{clientNo}, \text{propertyNo} \rightarrow \text{rentStart}, \text{rentFinish}$ (Primary key)
- fd2 $\text{clientNo} \rightarrow \text{cName}$ (Partial dependency)
- fd3 $\text{propertyNo} \rightarrow \text{pAddress}, \text{rent}, \text{ownerNo}, \text{oName}$ (Partial dependency)
- fd4 $\text{ownerNo} \rightarrow \text{oName}$ (Transitive dependency)
- fd5 $\text{clientNo}, \text{rentStart} \rightarrow \text{propertyNo}, \text{pAddress}, \text{rentFinish}, \text{rent}, \text{ownerNo}, \text{oName}$ (Candidate key)
- fd6 $\text{propertyNo}, \text{rentStart} \rightarrow \text{clientNo}, \text{cName}, \text{rentFinish}$ (Candidate key)

Example

Using approach 1

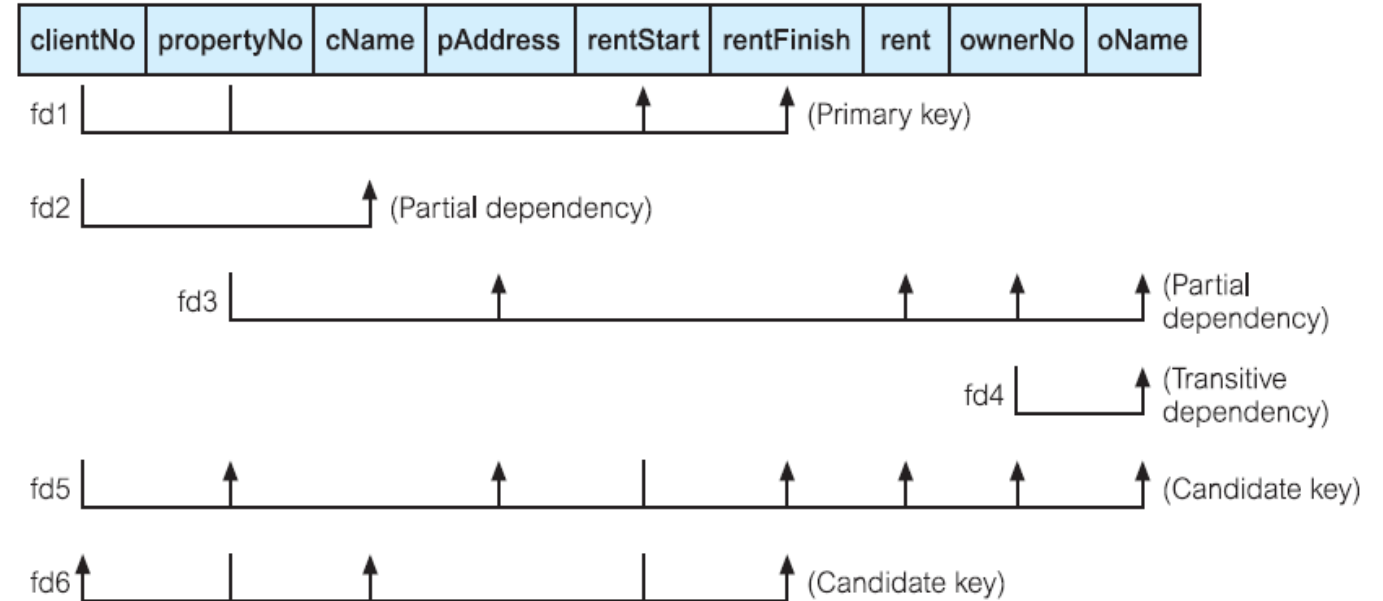
- Use the functional dependencies to **identify candidate keys** for the *ClientRental* relation.
Composite keys: (clientNo, propertyNo), (clientNo, rentStart), and (propertyNo, rentStart).
- Next, **select the primary key** for the relation: (clientNo, propertyNo)
 - For clarity, place the attributes that make up the primary key together at the LHS of the relation.
 - Now, the **ClientRental** relation is defined as follows:
ClientRental (clientNo, propertyNo, cName, pAddress, rentStart, rentFinish, rent, ownerNo, oName)
- The 1NF relation would be subject to the **update anomalies**. To remove some of these, you must transform the relation into 2NF.

1NF to 2NF

- 1NF to 2NF involves the removal of partial dependencies.
- Remove the partially dependent attribute(s) from the relation by placing them in a new relation along with a copy of their determinant.

Primary Key : (clientNo, propertyNo)

ClientRental



fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd2	clientNo → cName	(Partial dependency)
fd3	propertyNo → pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo → oName	(Transitive dependency)
fd5	clientNo, rentStart → propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart → clientNo, cName, rentFinish	(Candidate key)

1NF to 2NF

Primary Key : (clientNo, propertyNo)

- The attribute **cName** is partially dependent on the primary key (depends on only the **clientNo** attribute [see fd2]).
- The property attributes (**pAddress, rent, ownerNo, oName**) are partially dependent on the primary key (depends on only the **propertyNo** [see fd3]).
- The property rented attributes (**rentStart** and **rentFinish**) are fully dependent on the whole primary key (see fd1).
- The identification of partial dependencies within the *ClientRental* relation indicates that the relation is not in 2NF.
- To transform the *ClientRental* relation into 2NF requires the creation of new relations so that the **non-primary-key attributes are removed, along with a copy of the part of the primary key on which they are fully functionally dependent.**

1NF to 2NF

ClientRental

clientNo	propertyNo	cName	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	PG4	John Kay	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
CR76	PG16	John Kay	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	PG4	Aline Stewart	6 Lawrence St, Glasgow	1-Sep-02	10-Jun-03	350	CO40	Tina Murphy
CR56	PG36	Aline Stewart	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
CR56	PG16	Aline Stewart	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw



Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

- The result is the creation of three new relations called **Client**, **Rental**, and **PropertyOwner**.
- These three relations are in 2NF as every non primary-key attribute is fully functionally dependent on the primary key of the relation.

The relations have the following form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

Example

Using approach 2:

- Using the functional dependencies identified earlier, you can identify a primary key for the *Client* and *PropertyRentalOwner* relations.
- The format of the resulting 1NF relations are as follows:

Client (clientNo, cName)

PropertyRentalOwner (clientNo, propertyNo, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

- The property attributes (pAddress, rent, ownerNo, oName) are partially dependent on the primary key (depends on only the propertyNo [see fd3]).
- The property rented attributes (rentStart and rentFinish) are fully dependent on the whole primary key (see fd1).

1NF to 2NF

Using approach 2:

- The identification of partial dependencies within the **PropertyRentalOwner** relation indicates that the relation is not in 2NF.
- To transform the **PropertyRentalOwner** relation into 2NF requires the creation of new relations so that the non-primary-key attributes are removed along with a copy of the part of the primary key on which they are fully functionally dependent.
- The result is the creation of three new relations called **Client**, **Rental**, and **PropertyOwner**.
- These three relations are in 2NF as every non primary-key attribute is fully functionally dependent on the primary key of the relation.
- The relations have the following form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyOwner (propertyNo, pAddress, rent, ownerNo, oName)

6.4. Third Normal Form (3NF)

Third Normal Form (3NF)

3NF: A relation that is in 1NF and 2NF and in which no non-primary-key attribute is **transitively dependent** on the primary key.

- Although 2NF relations have less redundancy than those in 1NF, they may still suffer from update anomalies.

Example: To update the name of an owner, **Tony Shaw** (ownerNo CO93), you have to update two tuples in the *PropertyOwner* relation.

- If you update only one tuple and not the other, the database would be in an inconsistent state.
- This update anomaly is caused by a transitive dependency.
- You need to remove such dependencies by progressing to 3NF.

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw

2NF to 3NF

- 2NF to 3NF involves the **removal of transitive dependencies**.
- If a transitive dependency exists, remove the transitively dependent attribute(s) from the relation by **placing the attributes(s) in a new relation along with a copy of the determinant**.

These dependencies have altered compared with the original functional dependencies.

fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd2	clientNo → cName	(Partial dependency)
fd3	propertyNo → pAddress, rent, ownerNo, oName	(Partial dependency)
fd4	ownerNo → oName	(Transitive dependency)
fd5	clientNo, rentStart → propertyNo, pAddress, rentFinish, rent, ownerNo, oName	(Candidate key)
fd6	propertyNo, rentStart → clientNo, cName, rentFinish	(Candidate key)



Client

fd2	clientNo → cName	(Primary key)
-----	------------------	---------------

Rental

fd1	clientNo, propertyNo → rentStart, rentFinish	(Primary key)
fd5'	clientNo, rentStart → propertyNo, rentFinish	(Candidate key)
fd6'	propertyNo, rentStart → clientNo, rentFinish	(Candidate key)

PropertyOwner

fd3	propertyNo → pAddress, rent, ownerNo, oName	(Primary key)
fd4	ownerNo → oName	(Transitive dependency)

2NF to 3NF

Resulted relations after 2NF →

- Client** (clientNo, cName)
- Rental** (clientNo, propertyNo, rentStart, rentFinish)
- PropertyOwner** (propertyNo, pAddress, rent, ownerNo, oName)

- All the non-primary-key attributes within the **Client** and **Rental** relations are functionally dependent on only their primary keys.
 - Those relations have no transitive dependencies and are therefore already in 3NF.
- All the non-primary-key attributes within the **PropertyOwner** relation are functionally dependent on the primary key, except **oName**, which is transitively dependent on **ownerNo** (see fd4).
 - To transform the **PropertyOwner** relation into 3NF, you must remove this transitive dependency by creating two new relations called **PropertyForRent** and **Owner**.

2NF to 3NF

- The new relations have the form:

PropertyForRent (propertyNo, pAddress, rent, ownerNo)

Owner (ownerNo, oName)

- The **PropertyForRent** and **Owner** relations are in 3NF as there are no further transitive dependencies on the primary key.

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

PropertyOwner

propertyNo	pAddress	rent	ownerNo	oName
PG4	6 Lawrence St, Glasgow	350	CO40	Tina Murphy
PG16	5 Novar Dr, Glasgow	450	CO93	Tony Shaw
PG36	2 Manor Rd, Glasgow	375	CO93	Tony Shaw



Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Summary of the 3NF relations

- The **ClientRental** relation has been transformed by the process of normalization into four relations in 3NF.

ClientRental (clientNo, propertyNo, cName, pAddress, rentStart, rentFinish, rent, ownerNo, oName)

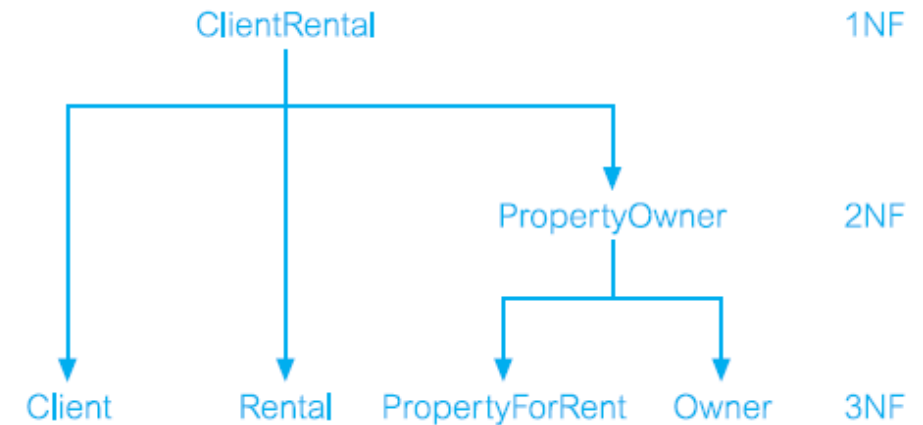
- The resulting 3NF relations have the form:

Client (clientNo, cName)

Rental (clientNo, propertyNo, rentStart, rentFinish)

PropertyForRent (propertyNo, pAddress, rent, ownerNo)

Owner (ownerNo, oName)



- The original **ClientRental** relation can be recreated by joining the **Client**, **Rental**, **PropertyForRent**, and **Owner** relations through the primary key/foreign key mechanism.

Summary of the 3NF relations

unnormalized table

ClientRental

clientNo	cName	propertyNo	pAddress	rentStart	rentFinish	rent	ownerNo	oName
CR76	John Kay	PG4	6 Lawrence St, Glasgow	1-Jul-03	31-Aug-04	350	CO40	Tina Murphy
		PG16	5 Novar Dr, Glasgow	1-Sep-04	1-Sep-05	450	CO93	Tony Shaw
CR56	Aline Stewart	PG4	6 Lawrence St, Glasgow	1-Sep-02	10-June-03	350	CO40	Tina Murphy
		PG36	2 Manor Rd, Glasgow	10-Oct-03	1-Dec-04	375	CO93	Tony Shaw
		PG16	5 Novar Dr, Glasgow	1-Nov-05	10-Aug-06	450	CO93	Tony Shaw

3NF tables

Client

clientNo	cName
CR76	John Kay
CR56	Aline Stewart

Rental

clientNo	propertyNo	rentStart	rentFinish
CR76	PG4	1-Jul-03	31-Aug-04
CR76	PG16	1-Sep-04	1-Sep-05
CR56	PG4	1-Sep-02	10-Jun-03
CR56	PG36	10-Oct-03	1-Dec-04
CR56	PG16	1-Nov-05	10-Aug-06

PropertyForRent

propertyNo	pAddress	rent	ownerNo
PG4	6 Lawrence St, Glasgow	350	CO40
PG16	5 Novar Dr, Glasgow	450	CO93
PG36	2 Manor Rd, Glasgow	375	CO93

Owner

ownerNo	oName
CO40	Tina Murphy
CO93	Tony Shaw

Activity 9

Consider the relation below and the given functional dependencies (FD1 to FD4).

Employee (*EmpID*, *DeptID*, *DeptName*, *ManagerID*, *ManagerName*, *Salary*)

FD1: $\text{EmpID}, \text{DeptID} \rightarrow \text{Salary}$

FD2: $\text{EmpID} \rightarrow \text{ManagerID}$

FD3: $\text{DeptID} \rightarrow \text{DeptName}$

FD4: $\text{ManagerID} \rightarrow \text{ManagerName}$

1. Identify the primary key of the *Employee* relation.

$\{\text{EmpID}, \text{DeptID}\}$

2. List down all full functional dependencies in the *Employee* relation.

$\text{EmpID} \rightarrow \text{ManagerID}, \text{DeptID} \rightarrow \text{DeptName}$

3. Write down the set of relation(s) resulting from the decomposition of the *Employee* relation into 2NF.

EmployeeWork(*EmpID*, *DeptID*, *Salary*)

Departments(*DeptID*, *DeptName*)

Managers(*EmpID*, *ManagerID*, *ManagerName*)

Activity 9

Consider the relation below and the given functional dependencies (FD1 to FD4).

Employee (*EmpID*, *DeptID*, *DeptName*, *ManagerID*, *ManagerName*, *Salary*)

FD1: *EmpID*, *DeptID* $\rightarrow$ *Salary*

FD2: *EmpID* $\rightarrow$ *ManagerID*

FD3: *DeptID* $\rightarrow$ *DeptName*

FD4: *ManagerID* $\rightarrow$ *ManagerName*

4. List down all transitive dependencies in the *Employee* relation.

(*ManagerID* $\rightarrow$ *ManagerName*)

5. Write down the set of relation(s) resulting from the decomposition of the *Employee* relation into 3NF.

EmployeeWork(*EmpID*, *DeptID*, *Salary*)

Departments(*DeptID*, *DeptName*)

Managers(*EmpID*, *ManagerID*)

ManagerDetails(*ManagerID*, *ManagerName*)

Pros & Cons in Normalization

Advantages

- Reduction of data redundancy within tables
- Reduce data storage space
- Reduce inconsistency of data
- Reduce update cost
- Remove many-to-many relationship
- Improve the flexibility of the system

Disadvantages

- Increase join
- Reduce efficiency
- Increase use of indexes
- Increase complexity of the system

Summary

- **UNF** is a table that contains **one or more repeating groups**.
- **1NF** is a relation in which the intersection of each row and column contains one and only one value.
- **2NF** is a relation that is in 1NF and every non-primary-key attribute is **fully functionally dependent** on the *primary key*.

Full functional dependency indicates that if A and B are attributes of a relation, B is fully functionally dependent on A if B is functionally dependent on A but not on any proper subset of A.

- **3NF** is a relation that is in 1NF and 2NF in which no non-primary key attribute is **transitively dependent** on the *primary key*.

Transitive dependency is a condition where A, B, and C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).

In-class Assignment 2 - Practical Assignment

Type: Online MCQ (Closed-Book)

Content: SQL

Date & Time: 25th February 2026 (Wednesday)

Location: Labs A, B, C, and D

	Time	Registration Number	Index Number
Group 1	1.00 – 2.00 p.m	2024cs001 to 2024cs116	24000019 to 24001163
Group 2	2.00 – 3.00 p.m	2024cs117 to 2024cs231	24001171 to 24002313

Important

- * Please note that there will be no make-up assignments for absent students.
- * Any student found using unauthorized materials or engaging in any form of academic misconduct will have their assignment invalidated and will be recorded as absent.

Any Questions?